ParaDEVS Library

Architecture and Interpretation Guide (library v0)

ParaDEVS Library workspace, Binghamton

October 9, 2026

Abstract

This document explains how the ParaDEVS library is built and how its results should be read. It is a companion to the **How-To Guide**, which covers *how to use* the library. Here we describe the layers and their contracts, how the engine drives MS4 models, the life of a branch from one event to the next, what each number in a result means under each selection rule, which guarantees from the theory hold and under which conditions, and where the design is meant to be extended. The theory referred to throughout is `main_paratemporal.tex` and `Paratemporal_Adaptation.tex` in `Do s/documents/paratemporal`.

ParaDEVS documents. The entry point is **Start Here**, a reading guide that links every document.

- **Build Report**: what has been built, how it was verified, what is next
- **How-To Guide**: setup, tutorials, API reference, troubleshooting
- **Architecture Guide** (this document): how the library works and how to read results
- **RL Plan**: reinforcement-learning plan, analysis and results
- **Control Spec**: action selection: Para.decide, graph-mode planning, value and policy iteration
- **Hybrid Study**: which draws to enumerate or sample, and how cost grows with lookahead
- **Fast-Copy Proposal**: why the engine replays steps, where time goes, how to speed it up

Contents

1 Purpose and design principles **3**

1.1 Step versus draw (principle P3 in detail) 3

2 Layered structure **5**

3 Integration with MS4 **6**

3.1 Driving an atomic model 6

3.2 Driving a coupled model: the ordered coordinator 6

3.3 Copying and restoring branches 7

3.4 Complete-state keys for merging 7

4 Execution model **7**
4.1 The records . . . 7
4.2 The main loop . . . 8
4.3 Life of a branch . . . 8
4.4 Suspension and replay . . . 8
4.5 What RunLocal does after an event completes . . . 9
4.6 Stopping . . . 9
4.7 Merging . . . 10

5 Interpreting results **10**
5.1 What the result is a distribution of . . . 10
5.2 By selection rule . . . 11
5.3 Total mass different from 1 . . . 11
5.4 Stats: reading cost, not results . . . 11
5.5 Summaries, marginals, and the old IH numbers . . . 11
5.6 Pitfalls when designing an experiment . . . 12

6 Guarantees and the conditions they need **13**
6.1 Model obligations . . . 13

7 Extension points **14**
7.1 Planned changes and where they go . . . 14

8 Verification record **14**

1 Purpose and design principles

The library turns an ordinary stochastic MS4 DEVS model into a *paratemporal* computation: whenever the model makes a random choice, the engine follows the possible outcomes and weights them, and it returns the resulting distribution over what the experiment observes. The design follows five principles.

P1. One model-side call. A model's only obligation is to obtain random values through `Para.choose(label, law)`. Otherwise it is an ordinary MS4 model, and it still runs under MS4's own simulator.

P2. The engine owns randomness. Models never decide how a choice is explored. The engine decides, through a *selection rule*: all outcomes, a sample, or a pruned subset. This is the paper's split between model and engine: "the model identifies the missing value and its conditional law, and the engine selects its possible values and assigns their weights."

P3. Branch at the draw. The engine branches each time the model calls `Para.choose`: one random value inside a DEVS transition. It does not branch on a whole "step", nor on a whole transition. The old tree engines treated a step, a draw, a batch of DEVS events and a tree level as the same thing. The library keeps them separate (Section 1.1).

P4. One clock. All branches advance on one simulated clock. Branching takes no simulated time (Adaptation, decision D1).

P5. Never trade correctness for size silently. Merging is *safe* (complete-state keys only), pruning is accounted for in an explicit residual, and combinations that defeat an estimator are refused.

1.1 Step versus draw (principle P3 in detail)

What the old engines did. In FinanceDEVS and BaseballDEVS, the unit of work was a *step*: one level of the tree. For each leaf and each outcome in a hard-coded list (`Buy/Sell/Holding`, `YrOut...Walk`), the engine rebuilt the coupled model, injected the outcome and the saved state by reflection, ran the simulation for several DEVS events (`Activate` → `GenerateCall` → the portfolio's external transition), and read the probability back out of a message. Four different things were bundled into that one unit:

Bundled into one old "step"	**What it really is**	**In the library**
one tree level (`GeneralTree.depth`)	exploration structure	No depth. A run ends by the stop rules: horizon, acceptor, or passive.
one random outcome, chosen *outside* the model from a fixed list	a draw	A call to `Para.choose` *inside* the model, wherever the randomness actually is.
one simulation run to a maximum time	several DEVS events	Time advances event by event on one clock. Many events can pass with no branching.
one save/restore of state (`GeneralNode` ↔ `setValue`)	a branch copy	An internal mechanism with no modelling meaning.

They could be treated as one because every old step contained exactly one random choice. As soon as a model has an event with no randomness, or an event with two random values, they come apart.

Draws are finer than transitions. One DEVS transition can contain several draws, and a later draw's law can depend on an earlier value. In the timed reward model a single external transition does

```
int qn = Para.choose("phase",  nextPhase(qPrev));       // P(q' | q)
int r  = Para.choose("reward", rewardLaw(qPrev, qn));   // P(r | q, q'): uses qn
```

- **Branching on the whole transition** (as in the V12 formalization) treats it as one random event with a joint law over next states, $P(s' \mid s, e, x)$. Here the model would have to supply the 4-outcome table $P(q', r \mid q)$, computed in advance.
- **Branching on each draw** (the current theory; Adaptation, conflict C1) splits at the first draw. Each child resumes the *same* transition with q' fixed and splits again at the second draw. The engine produces the same 4 children with weights $P(q' \mid q)\,P(r \mid q, q')$, and the model never states the joint law.

The same holds in coupled models: if two components are imminent at the same instant and each draws, that is one transition of the coupled model with two draws in two components. Draw-level branching handles it with no extra work.

When they coincide. If an event makes exactly one draw, branching on the draw and branching on the transition are the same. That is the case in the finance and baseball ports, so there P3 changes no numbers. It matters for dependent draws within one event, for simultaneous draws in coupled models, for draws in `initialize()`, and for events with no draw at all.

Why the theory insists on draws. A suspended branch only ever needs the conditional law of the *next* missing value, whose inputs are already known. That is what makes "copy the realized past, supply the next value, resume" correct (Adaptation, Proposition "Restart sufficiency"), and it is exactly what the library's replay does (Section 4.4).

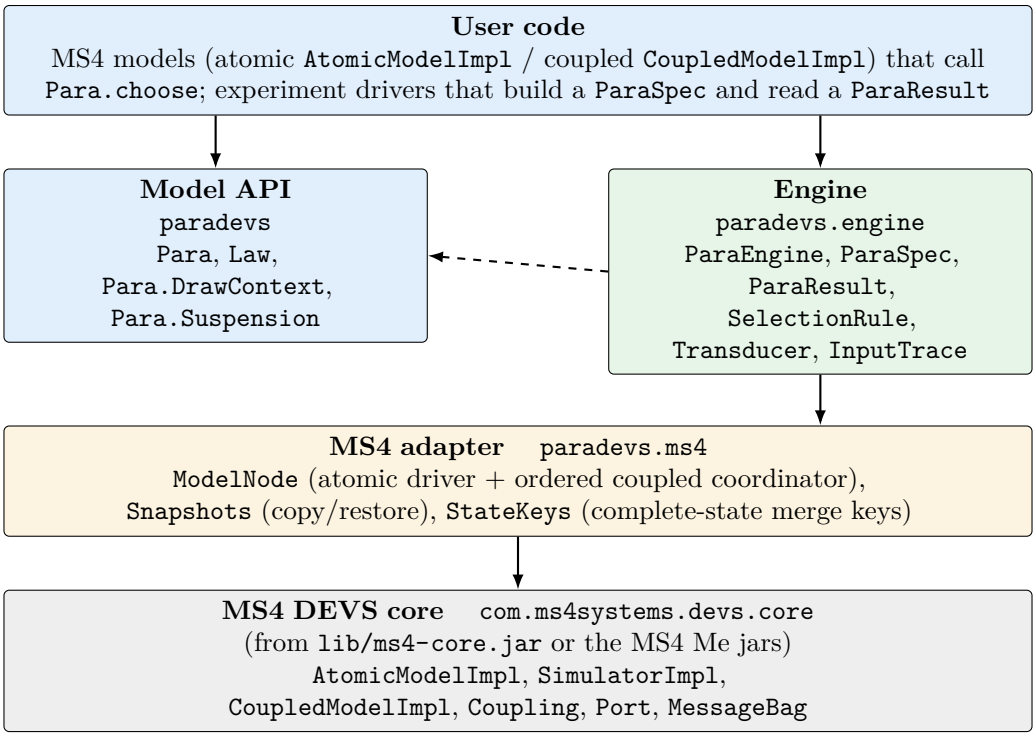


Figure 1: Layers and allowed dependencies (arrows point to what a layer uses). Models depend only on the Model API and on the MS4 core directly. The Model API knows nothing about the engine: the engine reaches it only by installing a thread-local draw context (dashed). The MS4 adapter is the only engine-side code that touches MS4 internals.

2 Layered structure

Layer	Classes (lines)	Responsibility and contract
Model API	`Law` (118), `Para` (101)	`Law`: a finite, ordered, validated discrete law (mass 1 within 10^{-6}, zero atoms dropped). `Para.choose`: samples when no engine is active; under the engine, returns the next supplied value or suspends the running transition. No dependency on the engine.
Engine	`ParaEngine` (285), `ParaSpec` (58), `ParaResult` (72), `SelectionRule` (70), `Transducer` (60), `InputTrace` (55), `OutputEvent`, `StopStatus`	The paper's algorithm: roots, Γ_{PT}, ADVANCE, doEXPANSION, RUNLOCAL, STOPSTATUS, merge, collector. Domain-free: knows nothing about finance or baseball.
MS4 adapter	`ModelNode` (156), `Snapshots` (47), `StateKeys` (111)	Runs one Parallel DEVS event on an MS4 model; copies and restores the complete model graph; computes the canonical complete state for merging.
MS4 core	external	Unchanged. Atomic models are driven through their own `SimulatorImpl`.

Why this split. The model API must be importable by any MS4 model, including DNL-generated code, without pulling in the engine, so that the same model runs under MS4 Me unchanged (P1). All MS4-specific behaviour (event times, couplings, serialization quirks) is isolated in the adapter, so the engine reads like the paper's pseudocode and could be pointed at another DEVS kernel by replacing one package.

3 Integration with MS4

3.1 Driving an atomic model

`ModelNode.Atomic` wraps a model and its own MS4 `SimulatorImpl` (every `AtomicModelImpl` creates one). One event is run with MS4's own methods, so model code that calls `getSimulator()` (for the current time, `holdIn` messages) behaves exactly as under MS4:

Event	**MS4 calls made**
initial	`sim.initialize(0)`: calls the model's `initialize()` (MS4 does this only at time 0), sets t_L, t_N
internal at t	`sim.computeOutput()` $\rightarrow$ outputs; `sim.executeNextEvent(t)`
external at t	`sim.processInput(t, bag)`: MS4 calls `externalTransition(e, bag)` with $e = t - t_L$
confluent at t	`sim.computeOutput()` $\rightarrow$ outputs; `sim.processInput(t, bag)`: MS4 calls `confluentTransition`

3.2 Driving a coupled model: the ordered coordinator

MS4's `CoordinatorImpl` routes messages through `HashMap`s keyed by simulator objects with identity hash codes. The order in which it processes simultaneous children can therefore differ between two copies of the same model. The engine replays transitions (Section 4.4), and a replay must request the same draws in the same order, so the library does not use MS4's coordinator. `ModelNode.Coupled` is a small Parallel DEVS coordinator with a fixed order:

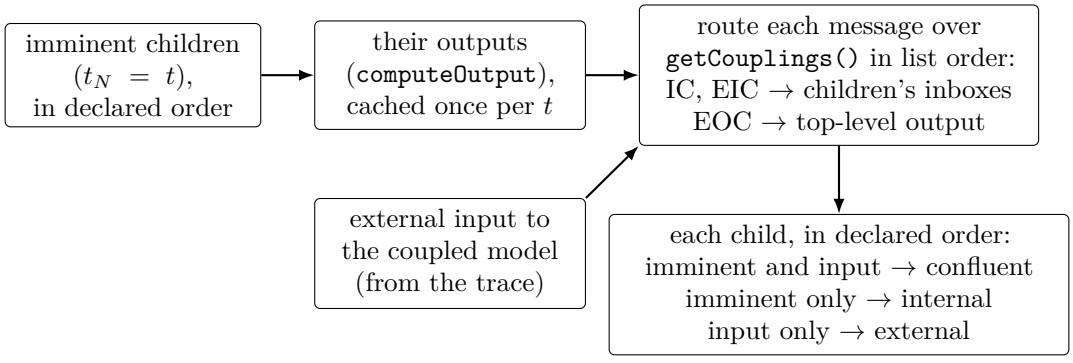


Figure 2: One event of a coupled model. Children and couplings are visited in their declared list order, which is the order of `addChildModel` and `addCoupling` calls (or the SES/PES-generated `make()`).

Nested coupled models are handled recursively. Atomic children are still driven by their own MS4 `SimulatorImpl`. Only the top model's outputs, reached through external output couplings,

are observed by the experiment.

3.3 Copying and restoring branches

A branch's model state is stored as the Java-serialized bytes of the *whole* model graph, including ports, MS4 simulators with their event times t_L, t_N, children and couplings (`Snapshots.save`). Copying a branch shares the bytes; running it restores an independent graph (`Snapshots.restore`). This is generic: any MS4 model whose fields are `Serializable` works, and nothing is read through model-specific accessors. It realizes the paper's restart requirement (Adaptation, Proposition "Restart sufficiency"): at a persistent boundary, the retained state and the event times are enough to continue with the correct law.

3.4 Complete-state keys for merging

`StateKeys.of(model)` builds a canonical value for the state. For each atomic model, in declared order, it records the class, name, MS4 t_L and t_N, and *every* instance field declared by the model's own classes, including bookkeeping such as `phase`, `sigma`, `currentTime`, `previousPhase`. Excluded are only structural references that are identical in every copy: ports, simulators, parent/child models, `PropertyChangeSupport`, MS4 helper options. Values are compared by `equals` when the class overrides it, otherwise by serialized bytes, otherwise by identity. Every fallback can only *prevent* a merge, never cause a wrong one. A model may supply its own key (`StateKeys.ParaStateKey`), and is then responsible for its completeness.

4 Execution model

4.1 The records

Record	Fields	Role (paper name)
`Branch`	state bytes, key, θ, t_N, j, n, ξ, w	A live persistent branch $k = (\eta, t, \theta, j, n, \xi, w)$. θ: time of its last persistent entry; t_N: model's next internal event; j: input-trace cursor; n: entry count; ξ: observation; w: weight.
`Continuation`	parent Branch, event kind and time, supplied values, pending draw, w	A departure suspended at a choice point: the payload ρ of a choice request.
`Request`	branch name; stop status *or* continuation	What one simulator step asks of Γ_{PT}.
`ParaResult`	collector, residual, stats	The collector $\widehat{\mu} = \sum_k w_k \delta_{(\xi_k, h_k)}$ and the omitted mass r.

Live branches are kept in a `TreeMap` by name. Requests are resolved in name order and new branches get fresh, increasing names, so a run is fully deterministic for a given seed.

4.2 The main loop

1. **Roots.** Serialize a fresh (uninitialized) model from the factory. Run its `initialize()` as the first departure under the selection rule, N times with weight $1/N$ each (Monte Carlo) or once with weight 1. Draws in `initialize()` branch like any other draw. Apply StopStatus, merge, install.
2. **Repeat while live branches remain:**
   (a) Γ_{PT}: resolve the requests from the previous step (stops are collected, choices expanded), remove their parents, merge the new children, install them.
   (b) Choose $t = \min_k \mathrm{NextTime}(k)$, where $\mathrm{NextTime}(k) = \min(t_N, \text{time of trace entry } j)$.
   (c) Advance every branch with NextTime $= t$, in name order. A branch that completes its event without a non-degenerate draw is updated in place; one that hits a draw or a stop condition files a request.

The event type is decided per branch: external if the next input comes first, internal if t_N comes first, confluent if they coincide.

4.3 Life of a branch

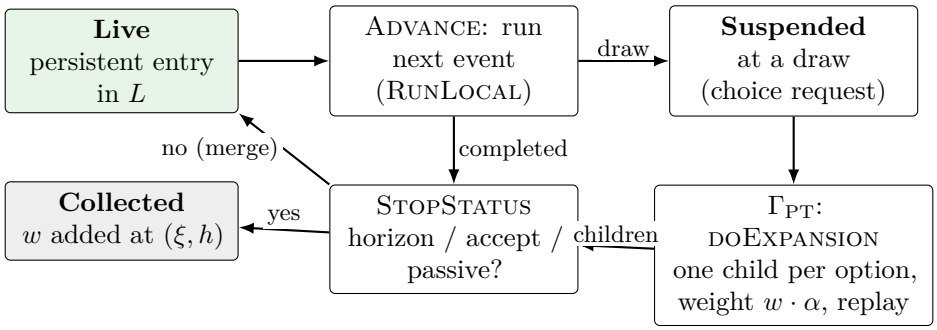


Figure 3: States of a branch. A suspended parent stays in L until Γ_{PT} replaces it, so its weight is always accounted for exactly once.

4.4 Suspension and replay

Java cannot pause a method in the middle and resume it later, so the library uses the paper's *derandomized departure* (Adaptation, Definition 5.1): an event is a deterministic function of the persistent state, the event arguments, and the drawn values $v_1, \ldots, v_N$.

Consequences:

- **Each requested value is written once and realized values are reused**, as RunLocal requires: earlier draws in the same event are replayed from the supplied list, never redrawn.
- **Determinism is checked**: a replayed draw whose supplied value is not in the law it now sees, or a replay that uses fewer values than supplied, raises an error naming the draw.
- **Laws with a single outcome never suspend** (RunLocal passes through degenerate draws).

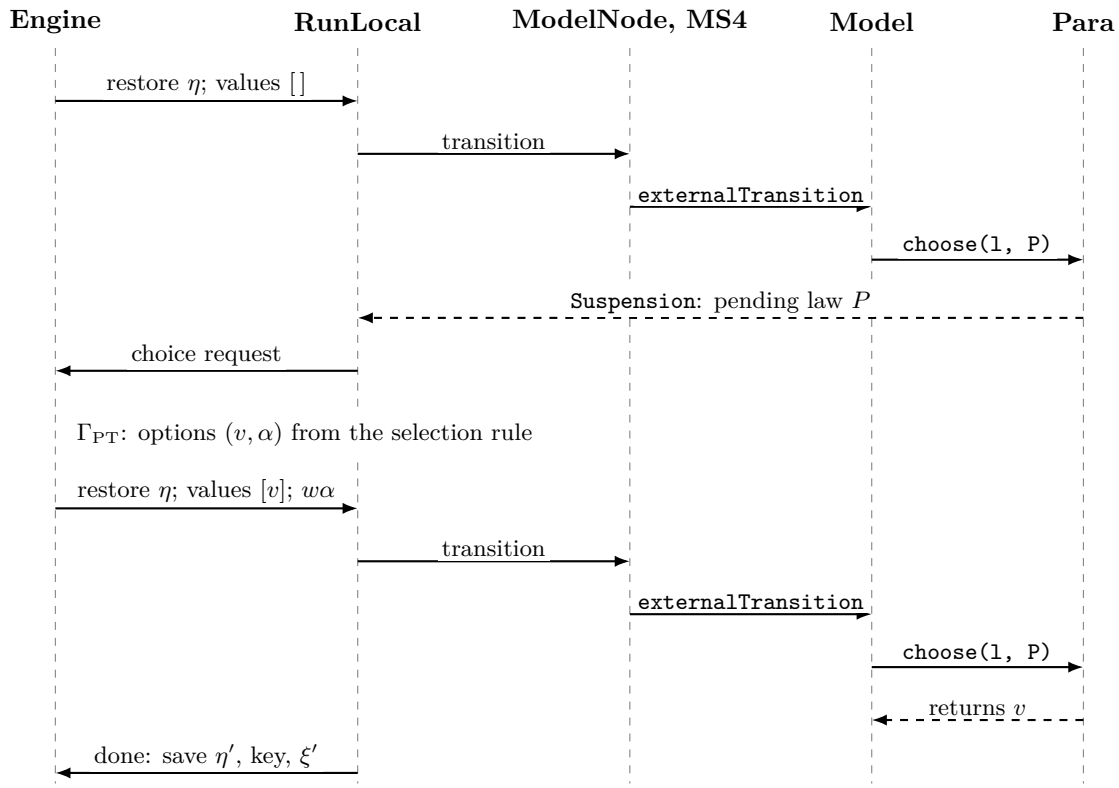


Figure 4: One draw, end to end. The first run discovers the pending law; each child re-runs the event from the same saved state with one more supplied value. A second draw in the same event repeats the pattern with two supplied values.

- **Cost**: an event with D draws and b options each is run $1 + b + b^2 + \cdots + b^D$ times. The statistic `departures` counts all runs, including replays.

4.5 What RunLocal does after an event completes

In this order: advance the trace cursor if an input was consumed; fold the event's outputs into ξ with the transducer (called for every completed event, with an empty list when there was no output); compute the state key (only when merging); serialize the model; record $\theta = t$, the new t_N, and $n + 1$.

4.6 Stopping

StopStatus is applied to roots and to every completed entry, in the paper's order:

1. `HORIZON` if $n \geq m$ (the initial state is entry 1);
2. `ACCEPT` if the acceptor returns true for (ξ, θ, n);
3. `PASSIVE` if $t_N = \infty$ and the trace is exhausted;
4. otherwise `LIVE`. An infinite time advance alone is not terminal while inputs remain.

4.7 Merging

Under `Merge.SAFE`, the children produced by one Γ_{PT} update (and the roots) are grouped by the key (state key, θ, j, n, ξ). Each group is replaced by its first member carrying the summed weight. This is the complete continuation key of the paper's Proposition "Safe merging", and merging stays local to the update, as the paper prescribes.

Expectation-only merging (opt-in). `Merge.EXPECTATION` is off by default and needs a transducer that implements `Transducer.Additive`, which splits $\xi = (G, \mathrm{rest})$ into an additive accumulated reward $G = \texttt{reward}(\xi)$ and the rest, with $\texttt{withReward}(\xi, r)$ replacing G. The key is (state key, $\theta, j, n,$ rest), with the reward zeroed. A group of weights w_i and rewards G_i becomes one record of weight $W = \sum_i w_i$ and reward A/W, $A = \sum_i w_i G_i$. The members share their future, so by linearity $E[G] = \sum_s A_s + \sum_s W_s\, E[G_{\mathrm{future}} \mid s]$ is unchanged, and so is $E[a(\mathrm{rest})\, G + b(\mathrm{rest})]$ for any functions a, b. The law of G is lost. The result therefore refuses `distribution()`, `marginal(g)` and the general `expectation(f)` with an `IllegalStateException`, and answers `expectation(slope, offset)`, which evaluates $\sum w\,(\mathrm{slope}(\mathrm{rest})\, G + \mathrm{offset}(\mathrm{rest}))$, and `expectedReward()`. Both functions receive only the rest, so the rule holds by construction. The transducer's contract: the increment that `record` adds to G, and the new rest, do not depend on G, and the acceptor ignores G. The mode is valid only for linear objectives with additive rewards; it is refused without `Additive` and with $N > 1$ roots.

5 Interpreting results

5.1 What the result is a distribution of

A `ParaResult` is a measure over pairs (ξ, h): the *observation* the transducer built from the top model's outputs, and the *stop status*. Three things follow.

- It is a distribution of what you chose to observe, at the moment each branch stopped. Internal state that never leaves the model through a port is invisible. Couple it out if you need it.
- Branches may stop for different reasons. Read the status: mixing `HORIZON` leaves (stopped mid-run) with `PASSIVE` leaves (finished) is legitimate but answers a mixed question. Filter by status, or design the experiment so that all branches stop for the same reason.
- Weights are exact path probabilities (exhaustive) or estimator weights (sampling). They are *not* frequencies of a single simulation.

5.2 By selection rule

Rule	What `distribution()` and `expectation(f)` are	Checks and uncertainty
Exhaustive	The exact terminal law and exact expectations (up to floating point, about 10^{-12} relative in the tests).	`totalMass()` $= 1$. A different value means a law in the model (or its data) is not proper; see below.
Partial ($p_{\min}$)	A lower bound: each event probability $P(E)$ satisfies $\widehat{\mu}(E) \le P(E) \le \widehat{\mu}(E) + r$ with $r =$ `residual()`.	`totalMass()` + `residual()` $= 1$. Expectations of bounded f are bracketed in the same way by $\pm r \cdot \sup \lvert f \rvert$.
Monte Carlo (N roots)	An unbiased estimate; each root contributes weight $1/N$ to one sampled outcome.	`totalMass()` $= 1$ exactly. Standard error of an event probability is $\sqrt{p(1-p)/N}$. Repeat with other seeds to see the spread.
Hybrid	Exact over the exhaustively treated draws, unbiased over the sampled ones.	As for Monte Carlo, on the sampled part.

5.3 Total mass different from 1

With proper laws and exhaustive selection the mass is 1. It differs only when a law was built with `Law.asGiven(..., tol)` from data whose rows do not sum to 1. Example: baseball's mass is 1.000131 because one source row sums to 1.0004. The result is then exactly what the data implies, which is what reproducing an old engine requires. For a probability statement about the data-generating process, either fix the data or divide by the mass. Both change the question slightly; record which one you used.

5.4 Stats: reading cost, not results

Stat	**Meaning**
`leaves`	Terminal branches collected (before equal outcomes are summed). `distribution().size()` is the number of distinct outcomes.
`choiceNodes`	Non-degenerate draws expanded (the paper's "random choice nodes").
`departures`	Event runs including replays: the main cost driver.
`merged`	Branches absorbed by merging (safe or expectation-only).
`maxLive`	Largest number of simultaneously live branches: the memory driver.

For the paper's timed reward example these are 64 leaves and 63 choice nodes, exactly as in the paper's companion program. With safe merging: at most 8 live branches and 27 choice nodes, with the same law.

5.5 Summaries, marginals, and the old IH numbers

`marginal(g)` pushes the distribution through a summary g; `expectation(f)` integrates a function. Both are *linear*, so any quantity the old IH engines produced by merging branches on a summary and adding their weights (or probability-weighted values) is exactly a marginal or expectation of the library's exact law. The acceptance tests reproduce them this way:

- finance: $P(\text{shares} = s)$ and $E[\text{cash} + s\,p_{\text{final}};\ \text{shares} = s]$ match the Demo Deliverable's IH output, and the expected net value is -38.9056847633 against the baseline's -38.9056847626;
- baseball: every positive-probability (bases, outs, total outs, runs) state after 5 and 6 plate appearances matches the original engine to 10^{-12}, and expected runs after 5 is 0.315529858502410.

The library therefore does not need IH merging to obtain IH results. The difference is that it never *discards* the information a merge would lose.

5.6 Pitfalls when designing an experiment

- **The horizon counts top-level events, including zero-time ones.** In a coupled model, an exchange such as "portfolio reports, trader reacts" is several entries at the same time. Prefer an acceptor on time ($t \geq T$) or let the model end passive, as the finance and baseball experiments do.
- **The observation is part of the merge key.** A full history (`Transducer.history()`) rarely merges. Keep only what the result needs: a count, a running total, the latest report.
- **Safe merging finds nothing when times differ.** Branches that reach the same state at different times (e.g. random service durations) are different continuations and are correctly kept apart.

Caution. Monte Carlo with $N > 1$ roots and `Merge.SAFE` (or `Merge.EXPECTATION`) is refused by the engine. The roots start identical, so merging would collapse the independent samples into one path. The expectation is preserved, as the theory says, but the estimate becomes a single trajectory. Use `Merge.NONE` with Monte Carlo.

6 Guarantees and the conditions they need

Guarantee	Source in the theory	Holds when
Exact law, mass 1, finite run	main_paratemporal, Theorem "Exact exhaustive result"	exhaustive selection; finite discrete laws; finitely many draws per event; the run ends (horizon, acceptor, or passive)
Unbiased estimates	main_paratemporal eq. (unbiased); Adaptation Prop. "Unbiasedness of sampled and hybrid expansion"	Monte Carlo, hybrid, or importance factors; N roots of weight $1/N$; no merging
Residual bounds	eq. (residual), Adaptation Prop. "Truncation bounds"	partial exhaustive selection
Merging changes nothing	Proposition "Safe merging"	exhaustive selection; complete key (Section 3.4); refused with $N > 1$ sampled roots
Expectation-only merging keeps expectations	linearity (RL plan, Section "Monte Carlo at every stage")	`Merge.EXPECTATION`; additive, inert reward (`Transducer.Additive`); statistics affine in the reward only; distribution queries refused
Order independence	Adaptation Prop. "Independence from the advancement order"	always (and the engine's order is fixed anyway)
Correct restart from a copy	Adaptation Prop. "Restart sufficiency"	the whole model state is serializable; no hidden external state

6.1 Model obligations

Every guarantee above rests on these properties of the model. The engine checks the first two at run time.

M1. Deterministic given draws. Given its state, the event, and the values returned by `Para.choose`, a transition (or `initialize`, or `getOutput`) always does the same thing and requests the same draws in the same order. No `Math.random`, clocks, I/O-dependent behaviour, or hash-order iteration over identity-hashed objects.

M2. Draws pass through. Code must not catch `Throwable`/`Error` around `Para.choose`.

M3. Finite discrete laws for exhaustive and partial selection. Continuous quantities must be sampled (Monte Carlo or hybrid), or the model changed to a discretized one; the theorems then describe that model.

M4. Complete, serializable state. Everything that influences the future lives in the model's fields and is `Serializable`. Use `transient` only for caches that can be rebuilt.

M5. Initialization at time 0, inside `initialize()`.

M6. Eager outputs (the paper's $ps\lambda$): the value emitted by `getOutput` is part of the state when the state is entered. Draws inside `getOutput` work mechanically, but they are the deferred (ps) pattern, whose guarantees additionally require output terminality. v0 does not check this.

7 Extension points

Extension point	Use
`SelectionRule`	New exploration strategies, e.g. importance sampling with a proposal law (return $(v, P(v)/Q(v))$), top-k pruning, or label-specific rules through `hybrid`. Must not depend on weights or other branches.
`Transducer`	What to observe and how compactly. Return immutable values with `equals/hashCode`.
`ParaSpec.Acceptor`	Experiment-specific stopping on (ξ, t, n).
`StateKeys.ParaStateKey`	A model-supplied canonical state key: faster, and lets a model normalize equivalent representations. The model is responsible for completeness.
`ModelNode`	The single point to port the engine to another DEVS kernel.

7.1 Planned changes and where they go

Change	Where it lands
Deferred outputs (ps)	RunLocal: draw the output at invocation for internal and confluent events (the "bridge"); merge key coarsened to $(x, R(z, \cdot))$ (Adaptation, Prop. "Lumped merging").
Reactive inputs (RL agent)	An input policy whose state is part of the branch and of the merge key, replacing the fixed `InputTrace`; its random choices become draws.
Parallel execution	doExpansion children and Advance of different branches are independent; only the merge in Γ_{PT} is a synchronization point.
Performance	Cache the restored model between Advance and its first replay; avoid reserializing unchanged children; incremental state keys.
MS4 Me IDE integration	Add the `paradevs` package to an MS4 Me project, with the MS4 Me jars as the core; place `Para.choose` in DNL code blocks (fully qualified names). Deferred; see the **How-To Guide**, Section 5.

8 Verification record

Test (checks)	Reference	What it establishes
Tutorial (10)	Hand calculation	Atomic and coupled models, input traces, Monte Carlo and plain MS4 runs agree.
Timed reward (24)	`pt_reward_enumeration.py`, exact rationals	The engine *is* the paper's algorithm: exact law, 64 leaves, 63 choice nodes; merging, Monte Carlo and residual bounds behave as proved; the refused combination is refused.
Finance (28)	Demo Deliverable baseline	A ported coupled model with feedback reproduces the old IH results as marginals.
Baseball (15)	Fresh run of the original BaseballDEVS_v7 engine	Same, in a second domain, with 39 and 55 states compared.

Run them with `bash scripts/build_library.sh` or the VS Code task. A change to the engine or the adapter is acceptable only if all of them still pass.